K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Lab Assignment-2

School of Engineering and Technology

Name = Tamanna

RollNo. = 2401201138

Program =BCA (AI & DS)

Operating System Lab ENCA252

Submitted to :- Jyoti Mam

Implementation of Banker's Algorithm for Deadlock Avoidance

Problem Statement

Problem Statement Deadlock is a situation in operating systems where a set of processes are unable to proceed because each is waiting for resources held by others. Banker's Algorithm is a deadlock avoidance technique that ensures the system remains in a safe state. In this assignment, students will implement Banker's Algorithm and determine whether the system is in a safe state, as well as find the safe sequence of execution.

Tools/Technology Used:

- **Python 3.12.3, Linux OS (Ubuntu 24.04 LTS via VirtualBox), nano**

Task 1 - System Input: The number of processes and resources were defined. Allocation matrix and Maximum matrix were taken as input from the user along with available resources.

```python
# Task 2: Need Matrix
need = []
for i in range(n):
    row = []
    for j in range(r):
        row.append(maxi[i][j] - alloc[i][j])
    need.append(row)

print("\n--- Need Matrix ---")
for i in range(n):
    print("P" + str(i) + ": " + str(need[i]))
```

Task 2 - Need Matrix:

The Need Matrix was calculated using the formula: Need = Maximum - Allocation. This represents the remaining resource requirement of each process.

```python
# Task 3 & 4: Safety Algorithm
work = avail[:]
finish = [False] * n
safe_seq = []

while len(safe_seq) < n:
    found = False
    for i in range(n):
        if not finish[i]:
            if all(need[i][j] <= work[j] for j in range(r)):
                for j in range(r):
                    work[j] += alloc[i][j]
                finish[i] = True
                safe_seq.append("P" + str(i))
                found = True
    if not found:
        break
```

Task 3 & 4 - Safety Algorithm Implementation:

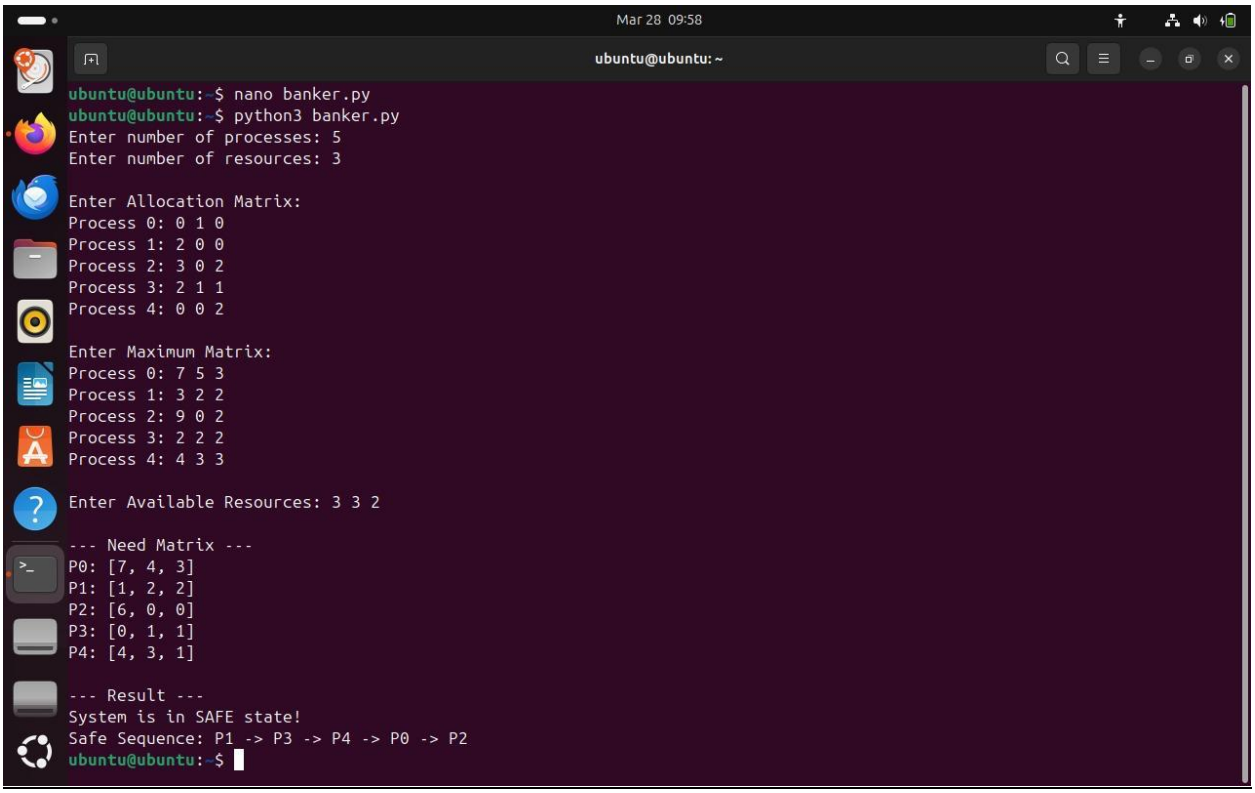
Work is initialized with Available resources. Each unfinished process is checked if its Need is less than or equal to Work. If yes, its resources are released and it is added to the safe sequence. If no process can proceed, the loop breaks indicating an unsafe state.

```python
# Task 5: Result
print("\n--- Result ---")
if len(safe_seq) == n:
    print("System is in SAFE state!")
    print("Safe Sequence: " + " -> ".join(safe_seq))
else:
    print("System is in UNSAFE state! Deadlock may occur.")
```

```
^G Help      ^O Write Out  ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File  ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

Task 5 - Result Analysis Code showing Safe/Unsafe state detection

If all processes are added to the safe sequence, the system is declared SAFE. Otherwise, it is UNSAFE and deadlock may occur.



Complete Program Output - Banker's Algorithm

The program successfully takes all inputs, calculates the Need Matrix, runs the Safety Algorithm and outputs the result. The system is in SAFE state with Safe Sequence: P1 → P3 → P4 → P0 → P2, confirming no deadlock will occur.